Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

# Raport
## la
## Lucrarea de laborator Nr. 5

*Disciplina: Tehnici de programare*

**Tema:** ”*Laborator Python: funcţii, metode şi importuri pe web*”

A efectuat : st. gr.SI – 262 Caragia Denis

A verificat : asistent universitar Emil Cebotaroș

Chișinău – 2026

## SARCINA LUCRĂRII

Scopul lucrării este de a învăța să folosesc Python pentru a comunica cu un server web și pentru a analiza răspunsurile primite de la el. Pe parcursul lucrării am urmărit patru lucruri: folosirea importurilor și a modulelor, diferența dintre atribute și metode, scrierea de funcții proprii și organizarea codului într-un modul reutilizabil. Rezultatul final este un program care primește adresa unui site și generează un raport despre el.

Când un browser sau un program vizitează o pagină, trimite serverului o cerere HTTP și primește un răspuns. Cererea și răspunsul conțin antete (headers), adică perechi nume–valoare cu informații suplimentare, cum ar fi tipul conținutului sau programul care a trimis cererea. Răspunsul are și un cod de stare: 200 înseamnă succes, 301 înseamnă redirecționare, 403 înseamnă acces interzis, iar 404 înseamnă pagină inexistentă.

În Python, un modul este un fișier cu cod care poate fi importat, un pachet este un folder de module, iar o bibliotecă este o colecție de module și pachete. Biblioteca requests este externă și se instalează cu pip, în timp ce urllib, json, ssl, socket și hashlib fac parte din biblioteca standard. O funcție este un bloc de cod cu nume care primește parametri și poate returna o valoare cu return. O metodă este o funcție care aparține unui obiect și se apelează cu punct, de exemplu response.json(), pe când un atribut este o valoare deja pregătită, precum response.status_code.

La toate cererile am respectat regulile laboratorului: am folosit un timeout, am făcut pauze de o secundă între cereri și am trimis către serverul cybercor.org doar cereri GET și HEAD.

## DESFĂȘURAREA LUCRĂRII

### Partea 1: Importuri

Am pregătit mediul de lucru cu Python 3.13, PyCharm și un mediu virtual, în care am instalat requests (versiunea afișată de requests.__version__ a fost 2.34.2). Am descărcat pagina principală în trei moduri de import: import requests, from requests import get și import requests as rq, toate returnând codul 200. Avantajul lui import requests este că se vede clar de unde vine fiecare funcție, iar from ... import scurtează codul. Aliasul este util pentru nume lungi, dar poate încurca atunci când numele ales nu spune nimic. Am descărcat pagina și cu

urllib.request, apoi am folosit dir() pentru a vedea ce conține modulul requests și help() pentru documentația funcției get. La exercițiul de măsurare a timpului, time.perf_counter() a dat aproximativ 0,3055 secunde, iar response.elapsed aproximativ 0,3016 secunde. Diferența apare pentru că primul măsoară tot timpul scurs în cod, iar al doilea doar intervalul dintre trimiterea cererii și primirea antetelor. La final, un bloc try / except ImportError verifică dacă biblioteca beautifulsoup4 este instalată și afișează un mesaj util dacă lipsește.

**Partea 2: Metode și atribute**

Am citit atribute ale răspunsului (status_code, ok, url, encoding) și am apelat metoda raise_for_status(), care nu face nimic la succes și ridică HTTPError la erori, folosită pe o pagină inexistentă care a dat 404. Am parcurs antetele cu response.headers.items() și am citit antete individuale cu .get(), constatând că numele antetelor nu țin cont de majuscule. Cu metode de șiruri (lower, count, find, splitlines, startswith) am numărat apariții, am extras titlul paginii, am numărat liniile HTML și am verificat dacă adresa folosește HTTPS. Am urmărit lanțul de redirecționări prin response.history, am comparat cererile HEAD și GET (HEAD returnează doar antetele, fără corp), am verificat cookie-urile și am folosit un obiect Session cu un antet User-Agent propriu, pe care l-am confirmat prin serviciul httpbin.org.

**Partea 3: Funcții**

Aici am scris funcții proprii, fiecare cu docstring. fetch() returnează răspunsul pentru o adresă și are timeout ca parametru implicit. get_status() returnează doar codul de stare, iar get_title() extrage titlul dintr-un text HTML fără a folosi rețeaua, ceea ce separă descărcarea de analiză. Cu adnotări de tip am observat că Python nu le verifică la rulare: apelul cu un număr în loc de adresă nu a fost oprit de adnotare, eroarea a venit din requests. page_exists() returnează True sau False și prinde erorile de rețea, check_paths() returnează un dicționar {cale: cod}, iar get_header() folosește un parametru implicit și argumente cu nume. security_headers() verifică cinci antete de securitate, iar score_headers() transformă rezultatul într-un scor de forma x/5. Perechea fetch_robots() și disallowed_paths() citește robots.txt și extrage căile interzise, tratând și cazul în care fișierul lipsește. În final, response_times(*urls) folosește argumente variabile, iar log(message, **details) folosește argumente cu nume variabile.

**Partea 4: Module din biblioteca standard**

Cu urllib.parse am descompus o adresă în schemă, domeniu, cale, parametri și fragment (urlparse) și am transformat legăturile relative în adrese complete (urljoin). Funcția extract_links() găsește legăturile cu o expresie regulată din modulul re și elimină duplicatele, iar split_links() le împarte în interne și externe, comparând domeniul. Am scris apoi clasa ImageFinder, moștenită din HTMLParser, care colectează atributul src al etichetelor <img>; parserul apelează singur metoda handle_starttag() de fiecare dată când întâlnește o etichetă de deschidere, iar testul cu assert a trecut. Cu hashlib am calculat amprenta SHA-256 a unei pagini, care se schimbă dacă pagina conține conținut dinamic. Am salvat și citit antetele într-un fișier JSON cu modulul json, am aflat adresa IP a unui domeniu cu socket.gethostbyname() și am calculat câte zile mai sunt până la expirarea certificatului SSL, folosind ssl și socket.

**Partea 5: Modulul webtools și programul principal**

Funcțiile reutilizabile au fost adunate în modulul webtools.py, importat apoi de main.py în trei feluri: import webtools, from webtools import ... și cu o constantă comună, DEFAULT_HEADERS, care conține antetul User-Agent trimis la toate cererile. Blocul if __name__ == "__main__": din webtools.py rulează un autotest doar atunci când fișierul este pornit direct, nu și când este importat, pentru că în cazul importului numele modulului este webtools, nu __main__. Programul main.py citește adresa site-ului din linia de comandă cu argparse și apelează funcțiile modulului. Funcția save_csv() scrie un fișier report.csv cu coloanele path, status și checked_at, iar site_report() colectează codul de stare, titlul, adresa IP, redirecționările, scorul antetelor de securitate, zilele rămase până la expirarea certificatului, numărul de legături interne și externe și căile interzise din robots.txt. Raportul este afișat în terminal și salvat în report.json. Ca exercițiu bonus am rescris get_title() și extract_links() cu biblioteca BeautifulSoup.

Probleme întâmpinate și rezolvări

Prima problemă a apărut la descărcarea paginii cu urllib, care a returnat eroarea CERTIFICATE_VERIFY_FAILED (certificat expirat), deși requests funcționa fără

probleme. Cauza este că urllib folosește magazinul de certificate din Windows, pe când requests are propriul set de certificate. Am rezolvat-o creând un context SSL cu lista de certificate din biblioteca certifi, fără a dezactiva verificarea, care ar fi nesigură. A doua problemă a fost eroarea 403 (Forbidden), apărută după ce certificatul a fost acceptat: serverul a respins identificatorul implicit al lui urllib. Am trimis un antet User-Agent propriu, printr-un obiect Request. A treia a fost o greșeală de scriere în lista de căi ("robots.txt" fără slash la început), care a produs adresa inexistentă cybercor.orgrobots.txt, deoarece funcția lipește calea direct după adresa site-ului. În final, la rularea lui main.py din PyCharm lipsea argumentul cerut de argparse; l-am adăugat în configurația de rulare.

**Rezultate**

Toate cele cinci părți rulează fără erori, iar cererile către cybercor.org returnează codul 200. Programul main.py generează raportul report.json și fișierul report.csv, care conține codurile de stare pentru căile verificate. Comparația de la exercițiul bonus a arătat că versiunile cu BeautifulSoup nu sunt mult mai scurte, dar sunt mai sigure. Pe un HTML cu majuscule, ghilimele simple și atribute în eticheta <title>, funcțiile bazate pe șiruri și expresii regulate au returnat un titlu greșit și au pierdut o legătură, pe când cele cu BeautifulSoup au returnat rezultatul corect.

## CONCLUZIE

Lucrarea m-a ajutat să înțeleg cum comunică un program Python cu un server web și cum se organizează codul în funcții și module. Am văzut diferența dintre biblioteca standard și bibliotecile externe, dintre atribute și metode, și am învățat să tratez erorile de rețea fără ca programul să se oprească. Problemele cu certificatul și cu codul 403 mi-au arătat că un cod corect poate eșua din cauza mediului sau a regulilor serverului. Am învățat de asemenea că analiza HTML cu șiruri și expresii regulate este fragilă și că o bibliotecă specializată, precum BeautifulSoup, este alegerea mai sigură. Un modul precum webtools.py face codul reutilizabil în alte proiecte, iar regulile de politețe (timeout, pauze și un User-Agent sincer) sunt parte importantă a unui program care vizitează site-uri reale.